**FreeRTOS Scheduling, Semaphores, and Multi-Core**

Vance Borus 2201308, Sean Bubernak 2234086 5/26/2026

Assignment: Lab 4 Report



# Introduction

The purpose of this lab is to explore scheduling with FreeRTOS, a small real-time operating system that integrates task scheduling and other features for simultaneous task execution. We explore using FreeRTOS as a basis for a custom Shortest Time Remaining Scheduler, which executes tasks with the shortest remaining execution time. We also explore using Semaphores, which synchronize communication between tasks that have to share resources or information. Finally we look at multi-core execution to truly execute tasks in parallel.

# Methods and Techniques

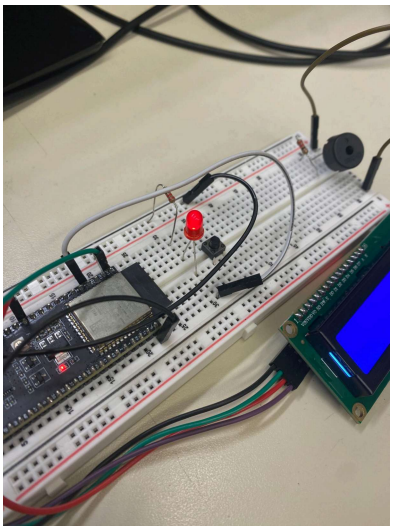
This lab was broken into two parts, one for Shortest Time Remaining Schedule and FreeRTOS, and using FreeRTOS with semaphores and multi-core. For the STR Scheduler, we introduced 3 tasks, plus our scheduler. FreeRTOS was responsible for executing ready tasks, while our scheduler managed which tasks were set to be ready, blocking all other tasks to ensure that only the shortest remaining ready task was available for FreeRTOS to run.

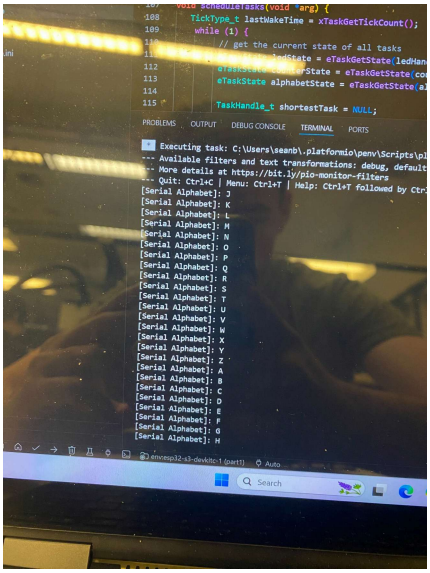
In the second part, we implemented 4 tasks that communicated via a Semaphore. One task pulled data from an external LCR sensor, and computed a SMA on a window of the data. This task would then give the semaphore, which would be consumed (taken) by two other tasks. One would scan for anomalies in the light value reading, while the other would print the current values to the screen. To load the processor in the background a prime number computation was set a low priority. We also split the processes between core 0 and 1.

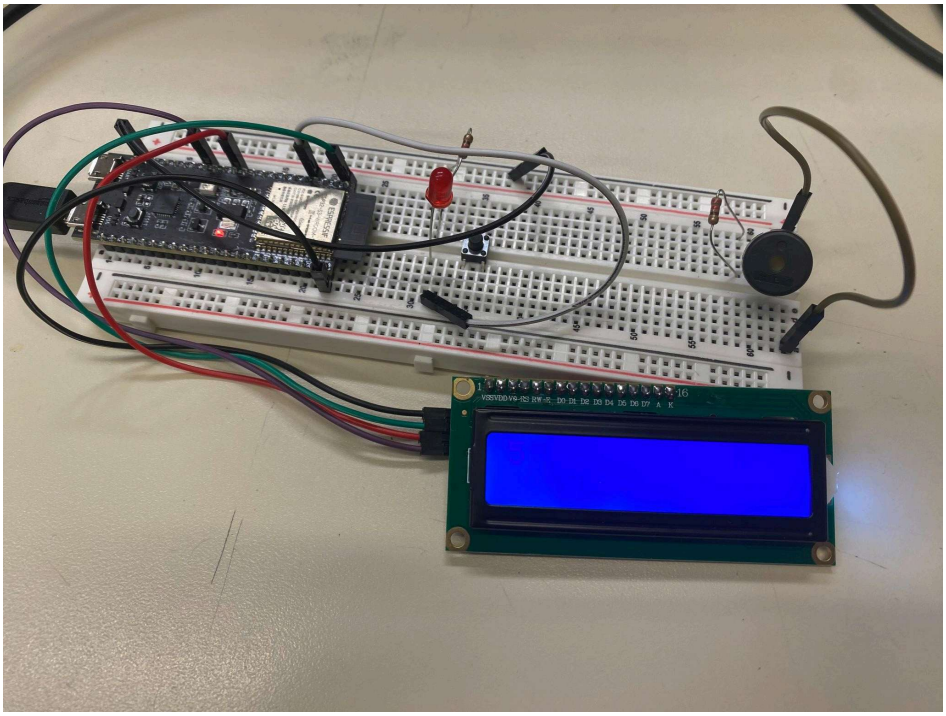
# Experimental Results

Describe specific test results which show how your code and hardware worked together to meet the lab requirements. Include 1-3 photos of your hardware setup.

## Part 1



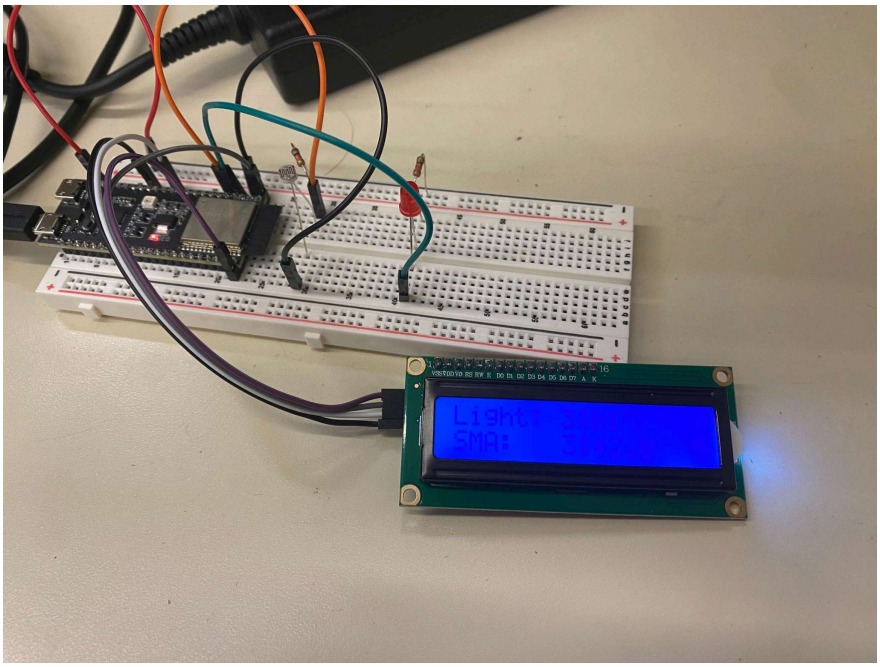


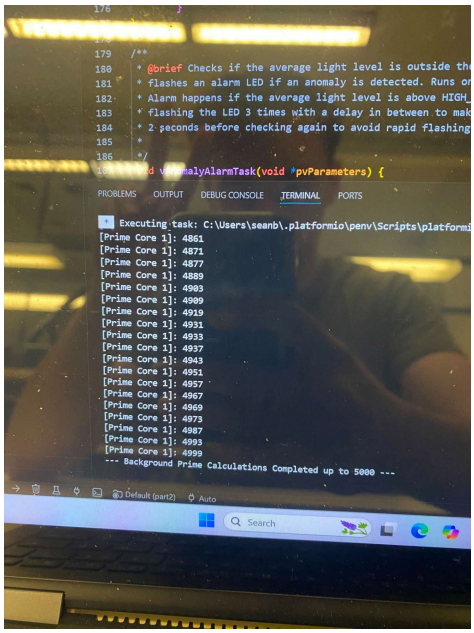



This part included implementing the scheduling of three tasks, pictured above (blinking the LED in a specific sequence, counting through the alphabet on the serial monitor and counting up to

20 on the LCD screen.Technically though, there was a fourth task, that being the shortest remaining time first scheduler. At the highest level we use FreeRTOS for scheduling but under the hood we have a task that is just a SRTF and handles the scheduling of the true tasks.

# Part 2






We were able to make our program with four tasks, those being the light detector, LCD, alarm, and prime calculation tasks. We also use semaphores to protect parts of our code while they

execute. The light detector is our semaphore giver while the LCD and alarm task are our semaphore consumers, the prime calculator is a background task.
It can be seen above that the LCD shows the current light value as well as a moving average of the light values.

# Overall Performance Summary

In both parts our schedulers were able to flawlessly display their simultaneous task operation without issue. Our shortest remaining time scheduler was able to properly prioritize tasks that had less time remaining, and our semaphores in part 2 ensured that data that was being displayed/read by other tasks was not stale.

# Teamwork Breakdown

In this lab we worked synchronously on each part but we worked on independent parts within them. For example in part 1 Sean made the LED, counter, and display tasks and setup while Vance handled the actual SRTF scheduler. In part 2 Sean prepped the modules, wrote the setup, LCD task, and prime calculation task while Vance wrote the light detector task and alarm task as well as several helper modules.

Github was very useful for this as we were able to work in our own branches and merge them easily since because we worked synchronously we could make sure merge conflicts were few and far between.

# Discussion and Conclusions

The most challenging part of this lab for us was part 1 because it felt like we were fighting against the natural flow of FreeRTOS by adding in our own SRTF under the hood. Ultimately though it was probably a good exercise to appreciate the beauty FreeRTOS. Things that required laborious programming to do such as this scheduler, were now very straightforward as we could see in part 2. Though the importance of protecting sections of code while we are using them we discovered was also crucial. Thus the introduction and usage of semaphores. Going forward into our final project we will absolutely be utilizing FreeRTOS and likely semaphores as well so this lab came at a good time.

# References

**[1]** PlatformIO, “*Quick Start — PlatformIO Core (CLI) Documentation*,” *PlatformIO Documentation*, 2014–present. Accessed: Apr. 21, 2026. [Online]. Available: https://docs.platformio.org/en/latest/core/quickstart.html

**[2]** Espressif Systems, *ESP32-S3-DevKitC-1 User Guide*. Espressif Documentation, 2016–2026. Accessed: Apr. 21, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32s3/esp32-s3-devkitc-1/index.html

**[3]** D. van Heesch, “Doxygen: Overview,” Doxygen Manual. Accessed: May 26, 2026. [Online]. Available: https://www.doxygen.nl/manual

# Appendix

## Total Number of hours spent: 15